Unidad 3 · Gestión del servidor

Tarea 3.3

Almacenamiento redundante, recurso de red y colas de impresión

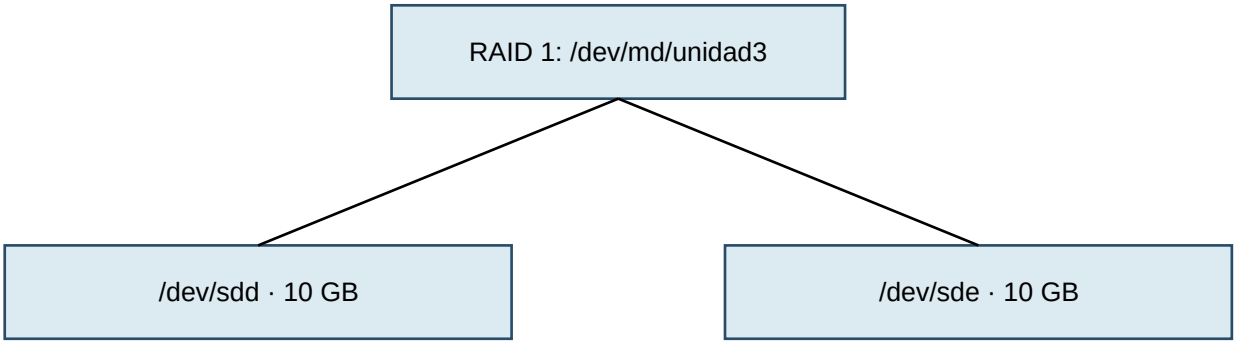
Reporte técnico del laboratorio
Ubuntu Server 26.04.1 LTS en VMware
Cuenta administradora: edwinmonte
8 de noviembre de 2026

Ensayo verificado: RAID 1 sano, degradación y reconstrucción, integridad SHA-256, recurso Samba montado mediante CIFS e impresión desde Windows por IPP. Cola virtual CUPS-PDF con prioridades 10 y 90.

Impresión simulada con salida PDF: no se utilizó una impresora física. Confirmar con el docente que acepta este destino virtual. Los registros y archivos de prueba son reales, recuperados del servidor por SSH. Horas de los logs: UTC.

1. Diseño del almacenamiento

Dispositivo	Función
sda, 25 GB	Sistema original de Ubuntu.
sdb / sdc, 10 GB cada uno	Datos con cuotas y respaldos de la tarea 3.2.
sdd / sde, 10 GB cada uno	Miembros nuevos del RAID 1.
/dev/md/unidad3	Arreglo RAID 1; alias observado /dev/md127.
/srv/raid_unidad3	Montaje ext4 persistente mediante UUID.



Ambos miembros contienen una copia de los datos. La capacidad útil observada fue aproximadamente 9.99 GiB, equivalente a un miembro, no a la suma. El arreglo no sustituye al respaldo ni ofrece independencia física si los discos virtuales residen en el mismo anfitrión.

2. Instalación y persistencia

El instalador comprobó tamaño, ausencia de firmas, particiones y montajes en sdd y sde. Creó el RAID con mdadm y metadatos 1.2, esperó la sincronización inicial, creó ext4 y añadió su UUID a /etc/fstab. Guardó la definición del arreglo en /etc/mdadm/mdadm.conf y actualizó initramfs. Se conservaron respaldos de las configuraciones previas bajo /root/unidad3.

3. Fallo lógico y reconstrucción

Se creó validacion_raid.txt y se calculó su SHA-256. Luego se marcó sde como fallido y se retiró del arreglo mediante mdadm. Esto simula un fallo lógico y una retirada del miembro; no se desconectó físicamente un disco desde VMware.

Etapa	Evidencia
Inicial	Estado clean, 2 dispositivos activos, [UU].
Degradado	Un miembro retirado, [U_]. Archivo original comprobado con SHA-256.
Continuidad	Se creó escritura_degradado.txt mientras faltaba un miembro.
Reconstrucción	Se reincorporó sde con --add y se esperó el fin de la recuperación.
Final	Estado clean, dos active sync, cero dispositivos fallidos. Integridad del archivo original: OK.

El registro ensayo_20261108_190030.log contiene la degradación y reconstrucción. cierre_3_3.log confirma el estado sano posterior y la suma correcta. El ensayo se detuvo más tarde en CUPS; ese error no afectó al RAID reconstruido.

4. Recurso Samba y montaje CIFS

Se creó el recurso Unidad3 sobre /srv/raid_unidad3/compartido, con acceso autenticado del usuario de servicio u3red. Su contraseña aleatoria se conserva únicamente en /root/unidad3/samba.credentials, con modo 600. Ese archivo no se incluye en el repositorio.

Samba se vinculó a lo y ens33, restringido a localhost y la subred privada del laboratorio. Se montó //127.0.0.1/Unidad3 en /mnt/unidad3_red mediante CIFS 3.0 y se verificó que la escritura aparece en el directorio compartido. Se añadió una entrada persistente con automontaje y dependencia del servicio smbd.

Alcance de la prueba SMB: cliente y servidor ejecutados en el mismo Ubuntu a través de loopback. Se demostró el montaje de red y la escritura por SMB; no se verificó un montaje de este recurso desde Windows. La prueba entre equipos se realizó para impresión.

5. Servidor de impresión y controlador

Se instalaron CUPS y printer-driver-cups-pdf. La cola U3_PDF utiliza cups-pdf:/ y el modelo lsb/usr/cups-pdf/CUPS-PDF_noopt.ppd. Se habilitó su compartición mediante cupsctl y printer-is-shared=true.

CUPS informó que los controladores PPD están obsoletos y dejarán de funcionar en una versión futura. En este laboratorio el controlador instalado funcionó y produjo PDF. Una evolución del diseño debe considerar impresión sin controlador cuando exista un destino IPP compatible.

Elemento	Configuración
Protocolo	IPP sobre TCP, puerto 631.
Cliente real	Windows 11; cliente Python con biblioteca estándar.
Servidor	Ubuntu, dirección observada 192.168.246.134.
Cola	U3_PDF.
Destino	/home/edwinmonte/PDF.
Trabajos	Red_Windows_PDF (50), Prioridad_baja (10), Prioridad_alta (90).

6. Colas, prioridades y resultados

El cliente Windows envió tres documentos PostScript por IPP y consultó los atributos job-id, job-priority y job-state. Los dos trabajos de prioridad se enviaron retenidos. En Ubuntu se pausó la cola, se liberaron ambos y se reactivó la impresora.

El log administrativo muestra que Job 3, prioridad 90, inició su backend a las 19:08:05 UTC y terminó a las 19:08:06. Job 2, prioridad 10, inició su backend después, a las 19:08:06, y terminó en ese segundo. El orden registrado coincide con la prioridad configurada.

```
I [08/Nov/2026:19:08:05 +0000] [Job 3] Started backend /usr/lib/cups/backend/cups-pdf (PID 12021)

I [08/Nov/2026:19:08:06 +0000] [Job 3] Job completed.

I [08/Nov/2026:19:08:06 +0000] [Job 2] Started backend /usr/lib/cups/backend/cups-pdf (PID 12028)

I [08/Nov/2026:19:08:06 +0000] [Job 2] Job completed.
```

7. Salidas de impresión y cierre

Prioridad_alta.pdf: 1 página. Texto recuperado: Prioridad_alta - Unidad 3.3

Prioridad_baja.pdf: 1 página. Texto recuperado: Prioridad_baja - Unidad 3.3

Red_Windows_PDF__DF.pdf: 1 página. Texto recuperado: Red_Windows_PDF - Unidad 3.3

Los tres documentos fueron comprobados como PDF no vacíos. La cola quedó habilitada y se desactivó el diagnóstico detallado de CUPS. La autenticación administrativa y la configuración Samba permanecen en el servidor del laboratorio.

8. Incidencias resueltas

El primer intento de pausa devolvió server-error-internal-error. Una continuación que especificó el motivo de pausa logró detener la cola y procesar los trabajos. No se determinó una causa raíz concluyente del error inicial.

La continuación esperaba un sufijo de archivo que no correspondía a los dos PDF de prioridad. La impresión había finalizado: se corrigieron los nombres esperados y el cierre verificó los archivos reales. Estos incidentes se conservan en los logs como diagnóstico y no se presentan como fallos del RAID o de los trabajos completados.

9. Referencias

https://openprinting.github.io/cups/doc/sharing.html

https://openprinting.github.io/cups/doc/options.html

https://openprinting.github.io/cups/doc/admin.html

Anexo · mdadm_unidad3.conf

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
ARRAY /dev/md/unidad3 metadata=1.2 UUID=7df7fe35:dea18eed:21a29b4a:28a6c28f
```

Anexo · smb.conf

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
[global]
    workgroup = WORKGROUP
    server role = standalone server
    security = user
    map to guest = Never
    interfaces = lo ens33
    bind interfaces only = yes
    hosts allow = 127.0.0.1 192.168.246.0/24
    server min protocol = SMB2

[Unidad3]
    path = /srv/raid_unidad3/compartido
    browseable = yes
    read only = no
    valid users = u3red
    create mask = 0660
    directory mask = 2770
```

Anexo · jobs_ipp.json

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.


```
[
  {
    "job-id": 1,
    "job-name": "Red_Windows_PDF",
    "job-priority": 50,
    "job-state": 5,
    "job-originating-user-name": "edwinmonte"
  },
  {
    "job-id": 2,
    "job-name": "Prioridad_baja",
    "job-priority": 10,
    "job-state": 4,
    "job-originating-user-name": "edwinmonte"
  },
  {
    "job-id": 3,
    "job-name": "Prioridad_alta",
    "job-priority": 90,
    "job-state": 4,
    "job-originating-user-name": "edwinmonte"
  }
]
```

Anexo · cierre_3_3.log

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
Verificacion final: 2026-11-08T19:11:11+00:00
/dev/md/unidad3:
           Version : 1.2
     Creation Time : Sun Nov  8 18:51:18 2026
        Raid Level : raid1
        Array Size : 10476544 (9.99 GiB 10.73 GB)
     Used Dev Size : 10476544 (9.99 GiB 10.73 GB)
      Raid Devices : 2
     Total Devices : 2
       Persistence : Superblock is persistent

       Update Time : Sun Nov  8 19:01:59 2026
             State : clean
    Active Devices : 2
   Working Devices : 2
    Failed Devices : 0
     Spare Devices : 0

Consistency Policy : resync

              Name : ubuntu:unidad3  (local to host ubuntu)
              UUID : 7df7fe35:dea18eed:21a29b4a:28a6c28f
            Events : 41

    Number   Major   Minor   RaidDevice State
       0       8       48        0      active sync   /dev/sdd
       2       8       64        1      active sync   /dev/sde
/srv/raid_unidad3/validacion_raid.txt: OK
TARGET           SOURCE              FSTYPE OPTIONS
/mnt/unidad3_red //127.0.0.1/Unidad3 cifs   rw,relatime,vers=3.0,cache=strict,upcall_target=app,user
    name=u3red,uid=0,noforceuid,gid=0,noforcegid,addr=127.0.0.1,file_mode=0755,dir_mode=0755,soft,no
    unix,serverino,mapposix,reparse=nfs,nativesocket,symlink=native,rsize=4194304,wsize=4194304,bsiz
    e=1048576,echo_interval=60,actimeo=1,closetimeo=1
la impresora U3_PDF está inactiva.  activada desde dom 08 nov 2026 19:08:06
U3_PDF-2                edwinmonte        1024   dom 08 nov 2026 19:08:06
U3_PDF-3                edwinmonte        1024   dom 08 nov 2026 19:08:06
U3_PDF-1                edwinmonte        1024   dom 08 nov 2026 18:56:03
PDF comprobado: Red_Windows_PDF__DF.pdf
PDF comprobado: Prioridad_alta.pdf
PDF comprobado: Prioridad_baja.pdf
ENSAYO_3_3_TERMINADO
```

Anexo · cerrar_3_3.sh

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
#!/bin/bash
# Recoge evidencia administrativa sin repetir las pruebas.
set -euo pipefail
umask 077
[[ $EUID -eq 0 ]] || { echo 'Ejecuta con sudo.'; exit 1; }
BASE=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)
cd "$BASE"
LOG=evidencias/cierre_3_3.log
install -o edwinmonte -g "$(id -gn edwinmonte)" -m 600 /dev/null "$LOG"
exec > >(tee "$LOG") 2>&1
echo "Verificacion final: $(date -Is)"
mdadm --detail /dev/md/unidad3
sha256sum -c evidencias/raid.sha256
findmnt /mnt/unidad3_red
lpstat -p U3_PDF -l
lpstat -W completed -o U3_PDF
for f in Red_Windows_PDF__DF.pdf Prioridad_alta.pdf Prioridad_baja.pdf; do
    test -s "/home/edwinmonte/PDF/$f"
    head -c 5 "/home/edwinmonte/PDF/$f" | grep -q '%PDF-'
    cp "/home/edwinmonte/PDF/$f" "evidencias/$f"
    chown edwinmonte:"$(id -gn edwinmonte)" "evidencias/$f"
    echo "PDF comprobado: $f"
done
grep -E '\[Job (2|3)\]' /var/log/cups/error_log | tail -n 250 > evidencias/cups_prioridades.log ||
    true
chown edwinmonte:"$(id -gn edwinmonte)" evidencias/raid.sha256 evidencias/cups_prioridades.log
chmod 600 evidencias/raid.sha256 evidencias/cups_prioridades.log
cupsctl --no-debug-logging
echo 'ENSAYO_3_3_TERMINADO'
```

Anexo · continuar_impresion_3_3.sh

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
#!/bin/bash
# Retoma solamente CUPS; conserva las pruebas RAID ya realizadas.
set -euo pipefail
umask 077
[[ $EUID -eq 0 ]] || { echo 'Ejecuta mediante sudo.'; exit 1; }
BASE=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)
cd "$BASE"
LOG="evidencias/impresion_$(date +%Y%m%d_%H%M%S).log"
install -o edwinmonte -g "$(id -gn edwinmonte)" -m 600 /dev/null "$LOG"
exec > >(tee "$LOG") 2>&1
echo "Inicio: $(date -Is)"
echo 'DIAGNOSTICO DEL ERROR ANTERIOR'
tail -n 100 /var/log/cups/error_log
echo 'REINTENTO DE PAUSA CON MOTIVO EXPLICITO'
if ! cupsdisable -r 'Prueba de prioridades Unidad 3' U3_PDF; then
    echo 'Reiniciando el planificador antes de repetir la pausa.'
    systemctl restart cups
    sleep 3
    cupsdisable -r 'Prueba de prioridades Unidad 3' U3_PDF
fi
trap 'cupsenable U3_PDF || true' EXIT
lpstat -p U3_PDF -l
baja=$(python3 -c 'import json; print(int(next(j["job-id"] for j in
    json.load(open("evidencias/jobs_ipp.json")) if j["job-priority"]==10)))')
alta=$(python3 -c 'import json; print(int(next(j["job-id"] for j in
```

Anexo · continuar_impresion_3_3.sh (continuación)

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
    json.load(open("evidencias/jobs_ipp.json")) if j["job-priority"]==90)))')
lp -i "U3_PDF-$baja" -H resume
lp -i "U3_PDF-$alta" -H resume
echo 'AMBOS TRABAJOS LISTOS, IMPRESORA AUN PAUSADA'
lpstat -o U3_PDF
cupsenable U3_PDF
for intento in $(seq 1 45); do
    if [[ -f /home/edwinmonte/PDF/Prioridad_alta.pdf && -f /home/edwinmonte/PDF/Prioridad_baja.pdf
    ]]; then break; fi
    sleep 1
done
for archivo in Red_Windows_PDF__DF.pdf Prioridad_alta.pdf Prioridad_baja.pdf; do
    f="/home/edwinmonte/PDF/$archivo"
    test -s "$f"
    head -c 5 "$f" | grep -q '%PDF-'
    cp "$f" "evidencias/$archivo"
    chown edwinmonte:"$(id -gn edwinmonte)" "evidencias/$archivo"
    stat -c 'PDF generado: %n (%s bytes)' "$f"
done
lpstat -W completed -o U3_PDF
grep -E "\\[Job ($baja|$alta)\\]" /var/log/cups/error_log | tail -n 180 >
    evidencias/cups_prioridades.log || true
chown edwinmonte:"$(id -gn edwinmonte)" evidencias/cups_prioridades.log
cupsctl --no-debug-logging
echo 'ENSAYO_3_3_TERMINADO'
```

Anexo · ensayar_3_3.sh

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
#!/bin/bash
# Ensayo de fallo logico/reconstruccion de RAID y prioridades de impresion.
set -euo pipefail
umask 077
[[ $EUID -eq 0 ]] || { echo 'Ejecuta mediante sudo.'; exit 1; }
BASE=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)
cd "$BASE"
install -d -o edwinmonte -g "$(id -gn edwinmonte)" -m 750 evidencias
LOG="evidencias/ensayo_$(date +%Y%m%d_%H%M%S).log"
install -o edwinmonte -g "$(id -gn edwinmonte)" -m 600 /dev/null "$LOG"
exec > >(tee "$LOG") 2>&1
MD=/dev/md/unidad3
nombre=$(basename "$(readlink -f "$MD")")
[[ $(cat "/sys/block/$nombre/md/level") == raid1 ]]
[[ $(cat "/sys/block/$nombre/md/degraded") == 0 ]]
test -e "/sys/block/$nombre/slaves/sde"
[[ $(findmnt -no SOURCE /srv/raid_unidad3) == "$(readlink -f "$MD")" ]]
echo "Inicio: $(date -Is)"
echo 'ESTADO SANO'
mdadm --detail "$MD"
printf 'Integridad RAID 1 - Unidad 3.3\n' > /srv/raid_unidad3/validacion_raid.txt
sha256sum /srv/raid_unidad3/validacion_raid.txt > evidencias/raid.sha256
sync
# Si el ensayo falla, intentar reincorporar el miembro retirado.
recuperar() {
    resultado=$?
    if [[ $(cat "/sys/block/$nombre/md/degraded") != 0 ]]; then
        echo 'Intentando reincorporar sde tras una interrupcion del ensayo.'
        mdadm "$MD" --add /dev/sde || true
    fi
    exit "$resultado"
}
trap recuperar EXIT
echo 'SIMULACION: FALLO LOGICO Y RETIRO DE SDE'
mdadm "$MD" --fail /dev/sde --remove /dev/sde
mdadm --detail "$MD"
cat /proc/mdstat
[[ $(cat "/sys/block/$nombre/md/degraded") == 1 ]]
sha256sum -c evidencias/raid.sha256
printf 'Escritura con RAID degradado\n' > /srv/raid_unidad3/escritura_degradado.txt
sync
echo 'REINCORPORACION Y RECONSTRUCCION'
mdadm "$MD" --add /dev/sde
cat /proc/mdstat
mdadm --wait "$MD" || true
```

Anexo · ensayar_3_3.sh (continuación)

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
[[ $(cat "/sys/block/$nombre/md/degraded") == 0 ]]
[[ $(cat "/sys/block/$nombre/md/sync_action") == idle ]]
mdadm --detail "$MD"
sha256sum -c evidencias/raid.sha256
grep -Fxq 'Escritura con RAID degradado' /srv/raid_unidad3/escritura_degradado.txt
echo 'OK: RAID reconstruido, lectura y escritura verificadas.'

echo 'RECURSO DE RED'
findmnt /mnt/unidad3_red
cat /mnt/unidad3_red/prueba_red.txt

echo 'COLA DE IMPRESION Y PRIORIDADES'
lpstat -p U3_PDF -l
lpstat -o U3_PDF
cupsctl --debug-logging
cupsdisable -r 'Prueba de prioridades Unidad 3' U3_PDF
# Ambos trabajos fueron enviados desde Windows por IPP, con prioridades 10 y 90.
baja=$(python3 -c 'import json; print(int(next(j["job-id"] for j in
    json.load(open("evidencias/jobs_ipp.json")) if j["job-priority"]==10)))')
alta=$(python3 -c 'import json; print(int(next(j["job-id"] for j in
    json.load(open("evidencias/jobs_ipp.json")) if j["job-priority"]==90)))')
lp -i "U3_PDF-$baja" -H resume
lp -i "U3_PDF-$alta" -H resume
lpstat -o U3_PDF
cupsenable U3_PDF
for intento in $(seq 1 45); do
    if [[ -f /home/edwinmonte/PDF/Prioridad_alta.pdf && -f /home/edwinmonte/PDF/Prioridad_baja.pdf
    ]]; then break; fi
    sleep 1
done
for archivo in Red_Windows_PDF__DF.pdf Prioridad_alta.pdf Prioridad_baja.pdf; do
    f="/home/edwinmonte/PDF/$archivo"
    test -s "$f"
    head -c 5 "$f" | grep -q '%PDF-'
    cp "$f" "evidencias/$archivo"
    stat -c 'PDF generado: %n (%s bytes)' "$f"
done
lpstat -W completed -o U3_PDF
grep -E "\\[Job ($baja|$alta)\\]" /var/log/cups/error_log | tail -n 120 >
    evidencias/cups_prioridades.log || true
cupsctl --no-debug-logging
chown edwinmonte:"$(id -gn edwinmonte)" evidencias/raid.sha256 evidencias/*.pdf
    evidencias/cups_prioridades.log
chmod 600 evidencias/raid.sha256 evidencias/*.pdf evidencias/cups_prioridades.log
echo 'ENSAYO_3_3_TERMINADO'
```

Anexo · instalar_3_3.sh

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
#!/bin/bash
# Instala RAID 1, recurso Samba autenticado y una cola virtual CUPS-PDF.
set -euo pipefail
umask 077
[[ $EUID -eq 0 ]] || { echo 'Ejecuta mediante sudo.'; exit 1; }
BASE=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)
cd "$BASE"
install -d -o edwinmonte -g "$(id -gn edwinmonte)" -m 750 evidencias config
LOG="evidencias/instalacion_$(date +%Y%m%d_%H%M%S).log"
install -o edwinmonte -g "$(id -gn edwinmonte)" -m 600 /dev/null "$LOG"
exec > >(tee "$LOG") 2>&1
echo "Inicio: $(date -Is)"
MD=/dev/md/unidad3
RAIZ=/srv/raid_unidad3
[[ ! -e $MD ]] || { echo 'El arreglo ya existe. Revisar antes de repetir.'; exit 1; }
for disco in /dev/sdd /dev/sde; do
    [[ -b $disco ]]
    [[ $(lsblk -nrpo TYPE "$disco") == disk ]]
    [[ $(blockdev --getsize64 "$disco") -eq 10737418240 ]]
    [[ -z $(lsblk -nro FSTYPE "$disco") ]]
    [[ -z $(wipefs -n "$disco") ]]
    [[ -z $(findmnt -nS "$disco") ]]
done
[[ $(findmnt -n -o SOURCE /srv/unidad3) == /dev/sdb ]]
[[ $(findmnt -n -o SOURCE /mnt/unidad3_respaldos) == /dev/sdc ]]
export DEBIAN_FRONTEND=noninteractive
apt-get update
apt-get install -y mdadm samba smbclient cifs-utils cups cups-client printer-driver-cups-pdf
install -d -m 700 /root/unidad3
mdadm --create "$MD" --metadata=1.2 --level=1 --raid-devices=2 --name=unidad3 /dev/sdd /dev/sde
    --run
echo 'Esperando sincronizacion inicial del RAID...'
mdadm --wait "$MD" || true
nombre=$(basename "$(readlink -f "$MD")")
[[ $(cat "/sys/block/$nombre/md/degraded") == 0 ]]
[[ $(cat "/sys/block/$nombre/md/sync_action") == idle ]]
```

Anexo · instalar_3_3.sh (continuación)

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
mdadm --detail "$MD"
mkfs.ext4 -F -L U3_RAID1 "$MD"
mkdir -p "$RAIZ"
uuid=$(blkid -s UUID -o value "$MD")
cp -p /etc/fstab /root/unidad3/fstab.antes_raid
printf 'UUID=%s %s ext4 defaults,nofail 0 2\n' "$uuid" "$RAIZ" >> /etc/fstab
mount "$RAIZ"
cp -p /etc/mdadm/mdadm.conf /root/unidad3/mdadm.conf.antes_raid
mdadm --detail --scan "$MD" > config/mdadm_unidad3.conf
cat config/mdadm_unidad3.conf >> /etc/mdadm/mdadm.conf
update-initramfs -u

echo 'CONFIGURACION SAMBA'
if ! getent passwd u3red >/dev/null; then
    useradd --system --user-group --no-create-home --shell /usr/sbin/nologin u3red
fi
mkdir -p "$RAIZ/compartido"
chown u3red:u3red "$RAIZ/compartido"
chmod 2770 "$RAIZ/compartido"
clave=$(od -An -N18 -tx1 /dev/urandom | tr -d ' \n')
printf '%s\n%s\n' "$clave" "$clave" | smbpasswd -s -a u3red
printf 'username=u3red\npassword=%s\n' "$clave" > /root/unidad3/samba.credentials
chmod 600 /root/unidad3/samba.credentials
unset clave
cp -p /etc/samba/smb.conf /root/unidad3/smb.conf.antes_unidad3
cat > config/smb.conf <<'SMB'
[global]
    workgroup = WORKGROUP
    server role = standalone server
    security = user
    map to guest = Never
    interfaces = lo ens33
    bind interfaces only = yes
    hosts allow = 127.0.0.1 192.168.246.0/24
    server min protocol = SMB2
```

Anexo · instalar_3_3.sh (continuación)

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
[Unidad3]
    path = /srv/raid_unidad3/compartido
    browseable = yes
    read only = no
    valid users = u3red
    create mask = 0660
    directory mask = 2770
SMB
testparm -s config/smb.conf
install -o root -g root -m 644 config/smb.conf /etc/samba/smb.conf
systemctl enable --now smbd
systemctl restart smbd
mkdir -p /mnt/unidad3_red
mount -t cifs //127.0.0.1/Unidad3 /mnt/unidad3_red -o
    credentials=/root/unidad3/samba.credentials,vers=3.0
printf '%s\n' '//127.0.0.1/Unidad3 /mnt/unidad3_red cifs credentials=/root/unidad3/samba.credentials
    ,vers=3.0,_netdev,nofail,x-systemd.automount,x-systemd.requires=smbd.service 0 0' >> /etc/fstab
printf 'Prueba de escritura mediante SMB\n' > /mnt/unidad3_red/prueba_red.txt
cmp /mnt/unidad3_red/prueba_red.txt "$RAIZ/compartido/prueba_red.txt"
findmnt /mnt/unidad3_red
echo 'OK: recurso Samba montado mediante CIFS y escritura verificada.'

echo 'CONFIGURACION CUPS'
systemctl enable --now cups
modelo=$(lpinfo -m | awk 'tolower($0) ~ /cups-pdf/ && !encontrado {print $1; encontrado=1}')
[[ -n $modelo ]] || { echo 'No se encontro el controlador CUPS-PDF.'; exit 1; }
echo "Controlador seleccionado: $modelo"
lpadmin -p U3_PDF -E -v cups-pdf:/ -m "$modelo"
cupsctl --share-printers
lpadmin -p U3_PDF -o printer-is-shared=true
lpstat -p U3_PDF -l
lpstat -v U3_PDF
cp /etc/cups/cupsd.conf config/cupsd.conf
chown -R edwinmonte:"$(id -gn edwinmonte)" config
echo 'INSTALACION_3_3_TERMINADA'
```

Anexo · prueba_ipp.py

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
"""Cliente IPP mínimo: envía documentos desde Windows a CUPS por TCP/631."""
import http.client, struct, json
from pathlib import Path
HOST='192.168.246.134'; URI=f'ipp://{HOST}:631/printers/U3_PDF'
def attr(tag,name,value):
    n=name.encode();v=value if isinstance(value,bytes) else value.encode()
    return bytes([tag])+struct.pack('!H',len(n))+n+struct.pack('!H',len(v))+v
def request(op,extra=b'',data=b''):
    attrs=attr(0x47,'attributes-charset','utf-8')+attr(0x48,'attributes-natural-
    language','en')+attr(0x45,'printer-uri',URI)+attr(0x42,'requesting-user-name','edwinmonte')
    body=struct.pack('!BBHI',1,1,op,1)+b'\x01'+attrs+extra+b'\x03'+data
    c=http.client.HTTPConnection(HOST,631,timeout=20)
    c.request('POST','/printers/U3_PDF',body,{'Content-Type':'application/ipp'})
    r=c.getresponse();payload=r.read();c.close()
    status=struct.unpack('!H',payload[2:4])[0] if len(payload)>=8 else -1
    if r.status!=200 or not 0<=status<0x100:raise RuntimeError(f'HTTP {r.status}, IPP {status:#x}:
    {payload!r}')
    values={};pos=8;last=''
    while pos<len(payload):
        tag=payload[pos];pos+=1
        if tag==3:break
        if tag<=0x0f:continue
        n=struct.unpack('!H',payload[pos:pos+2])[0];pos+=2
        name=payload[pos:pos+n].decode();pos+=n
```

Anexo · prueba_ipp.py (continuación)

Las líneas largas se ajustan para lectura. Ejecutar los archivos del paquete, no copiar desde el PDF.

```
        size=struct.unpack('!H',payload[pos:pos+2])[0];pos+=2
        value=payload[pos:pos+size];pos+=size
        if name:last=name
        values[last]=struct.unpack('!i',value)[0] if tag in (0x21,0x23) and len(value)==4 else
    value.decode(errors='replace')
    return values
jobs=[]
for title,priority,hold in
    [('Red_Windows_PDF',50,False),('Prioridad_baja',10,True),('Prioridad_alta',90,True)]:
    extra=attr(0x42,'job-name',title)+attr(0x49,'document-format','application/postscript')
    extra+=b'\x02'+attr(0x21,'job-priority',struct.pack('!i',priority))
    if hold:extra+=attr(0x44,'job-hold-until','indefinite')
    doc=('%!PS\n/Courier findfont 16 scalefont setfont\n72 720 moveto\n('+title+' - Unidad 3.3)
    show\nshowpage\n').encode()
    result=request(2,extra,doc)
    jobid=result['job-id']
    details=request(9,attr(0x21,'job-id',struct.pack('!i',jobid)))
    record={k:details.get(k) for k in ['job-id','job-name','job-priority','job-state','job-
    originating-user-name']}
    jobs.append(record);print(json.dumps(record,ensure_ascii=False))
(Path(__file__).resolve().parent/'jobs_ipp.json').write_text(json.dumps(jobs,indent=2),encoding='utf
    -8')
print('IPP_RED_VERIFICADO: documentos enviados desde Windows a Ubuntu por TCP/631.')
```